

● 오라클 1일차

1. 오라클 DBA 양성과정 97기 (5개월 과정)

- a. SQL, PLSQL, Linux, Administration, Backup & Recovery
- b. performance Tuning, SQL Tuning, RAC 등 교육 예정

1. 수업 진행 전 규칙 - 경청을 하자, 복습을 철저히 (내용 자체가 어렵다.), 수업시간에 딴 짓 x, 일기를 작성해라(?) - 작업 일지를 작성 했다는 것처럼 해봐라, 문해력을 키워라 (책을 많이 읽으면 도움이 될 것이다.)
2. 면접에서 비전공자, 전공자가 혹시 왜 DB 분야로 오게 되었는데 지에 대해 꼭 물어본다.

- a. xe softwar 설치, xe 데이터베이스 생성
- b. client(유저) - server(db 서버), 2-tier 환경
 - i. 개발은 2-tier 서버는 3-tier 등등
 - ii. client tool

1. SQL*Plus
2. SQL Developer
3. orange tool # 라이선스 필요
4. toad tool # 라이선스 필요

1. Run SQL Command Line 실행

2. conn sys / 비밀번호 as sysdba

3. select를 하고 받아올 툴이 필요 > SQLPlus

- a. 다양한 툴이 있지만 Oracle 같은 경우에는 > SQL Developer 사용

alter user hr identified by hr account unlock;

/*

계정에 대한 상태 정보가 EXPIRED & LOCKED 해결방법

패스워드 변경 및 unlock 해제

↳ hr은 사용자 데이터가 아니기에 DBA로 설정 x default 값으로 설정

*/

4. 오라클은 두가지 구조로 되어 있다. 메모리 ,디스크

- a. 디스크에 저장되어 있는 정보를 메모리에 올려 실행

5.

SQL(Sturctured Query Language)

데이터베이스에서 데이터를 정의, 조작, 제어하기 위해 사용하는 언어
ANSI(American National Standards Institute) 표준어인 SQL를 사용한다.

DQL(Data Query Language)

- 데이터베이스에 저장된 데이터를 조회하는 명령어
- SELECT

DML(Data Manipulation Language)

- 데이터베이스에 데이터를 입력, 수정, 삭제하는 명령어
- INSERT, UPDATE, DELETE, MERGE

TCL(Transaction Control Language)

- 데이터의 일관성을 유지하면서 안정적으로 데이터를 복구 시키기 위한 명령어
- COMMIT, ROLLBACK, SAVEPOINT

DCL(Data Control Language)

- 데이터베이스에 저장된 데이터를 관리하기 위하여 데이터의 보안성을 제어하기 위한 명령어(권한관리)
- GRANT, REVOKE

DDL(Data Definition Language)

- 데이터베이스 & 테이블 등을 생성, 삭제하거나 구조를 변경하기 위한 명령어
- CREATE, ALTER, DROP, TRUNCATE, RENAME, COMMENTS

- SELECT 문

- 데이터베이스에 저장된 데이터를 조회하는 명령어
- SELECT 컬럼이름, 컬럼이름...
- FROM 소유자명, 테이블명 ;
- Ex) select *
- from hr.employees;
 - 소유자명이 없을 경우 오류가 발생하기 때문에 소유자명을 확실하게 하거나 혹은 권한 부여를 제대로 해야 된다. (Key) 할당에 대한 정보

SELECT 문 기능

- projection : 열 추출
- select employee_id from hr.employees;
- select employee_id, last_name From hr.employees;
- selection
- join : 서로 다른 테이블에 있는 데이터를 추출

```
SELECT e.*, d.*
FROM hr.employees e, hr.departments d
WHERE employee_id = 100
AND e.department_id = d.department_id;
```

- SQL문 작성

- SQL문은 대소문자를 구분하지 않습니다.
- SQL문은 한줄 또는 여러줄에 입력할 수 있다.
- 키워드는 약어로 표시하거나 여러줄에 걸쳐 입력할 수 없다.
- 절은 대개 별도의 줄에 입력한다.
- 가독성을 위해 들여쓰기를 사용한다.
- SQL*PLUS에서는 SQL문이 반드시 세미콜론(;)으로 끝나야 한다.
- SQL Developer에서는 SQL문이 선택적으로 세미콜론(;)으로 끝낼 수 있다.
단 여러개의 SQL문을 작성하는 경우에는 꼭 세미콜론으로 끝난다.

- 산술연산자

- *, /, +, - 까지 존재
- number type : *, /, +, -
- char, varchar2 : 산술 연산자를 이용할 수 없다.
- date : +, -
- SELECT salary, salary * 12 FROM hr.employees;
- SELECT employee_id, hire_date, hire_date + 100 FROM hr.employees;
- # 산술연산자 우선순위 (동일한 순위일 경우에는 왼쪽 > 오른쪽 수행)
- Ex) SELECT a * b / c + d - e FROM dual;
 - ((a * b) / c) + d) - e
 - # 연산자 우선순위를 제어하는 방법은 괄호를 이용하면 된다.

- NULL 값의 정의

- NULL은 사용할 수 없거나, 할당되지 않거나, 알 수 없거나, 적용할 수 없는 값, 결측치

- NULL은 0이나 공백이 아니다.
- SELECT
 - employee_id,
 - salary,
 - commission_pct,
 - ((salary * 12) + (salary * 12 * commission_pct)),
 - ((salary * 12) + (salary * 12 * nvl(commission_pct,0)))
 - FROM hr.employees;
 - # nvl 함수는 NULL 값을 실제값으로 대체해주는 함수
 - if commission_pct is null then
 - 0;
 - else
 - commission_pct;
 - end if;
- # 열별칭
 - 열이름을 다른 이름으로 출력
 - 열이름 바로 뒤에 한칸 띄어쓰기 한 후 입력
 - 열별칭에 특수문자, 대소문자 구분, 공백을 하고 싶을 때는 “ “ 이용
 - 숫자가 먼저 나오면 “ “ 묶어 사용
- SELECT
 - employee_id 사번,
 - salary 급여,
 - commission_pct,
 - ((salary * 12) + (salary * 12 * commission_pct)) annual_salary_1,
 - ((salary * 12) + (salary * 12 * nvl(commission_pct,0))) as annual_salary_2
 - FROM hr.employees;
- # 연결연산자
 - 열이나 문자열을 다른 열에 연결한다.
 - 두개의 세로선(||)으로 표현
 - 결과 열은 문자열로 생성한다.
 - SELECT
 - employee_id
 - , last_name
 - , first_name
 - , last_name || first_name name
 - FROM hr.employees;
- # 리터럴 문자열
 - 문자, 숫자, 날짜를 리터럴 문자열로 표현하기 위해 문자열로 표현 (' ')
 - SELECT
 - '사번 : ' || employee_id
 - , last_name
 - , first_name
 - , last_name || ' ' || first_name name
 - FROM hr.employees;
 - SELECT

- '사원 번호는 ' || employee_id||'입니다.'
- FROM hr.employees;
- 대체 인용연산자(q)
 - 대괄호 안에 있는 것을 문자열로 이해 < >, [, (), { }, !! 가능
 - SELECT
 - q'[My name's]' || last_name||' ' ||first_name "이름"
 - FROM hr.employees;
- 중복 제거
 - 기본적으로 query(select)결과에는 중복행을 포함한 모든 행이 표시된다.
 - distinct, unique 키워드를 이용해서 중복 제거
 - SELECT절 제일 앞에서 한 번만 사용
 - SELECT
 - distinct department_id
 - FROM hr.employees;
 - SELECT
 - unique department_id
 - FROM hr.employees;